H2S-NN: Heterogeneous Heterophilic Sheaf Neural Network

Anonymous Author(s)

Abstract

Learning heterogeneous heterophilic graphs requires addressing two properties. First, heterogeneity represents distinct semantic roles through multiple node and relation types in graphs. Second, heterophily means that connected nodes may differ in labels or features, making similarity-based neighbor aggregation unreliable. Another important aspect is the use of meta-path schemas, which expose typed multi-hop semantics. Many existing meta-path methods represent concrete path instances through endpoint-level views and discard the internal path structure and neglect the higher-order information. Therefore, we introduce the Heterogeneous Heterophilic Sheaf Neural Network (H2S-NN), which retains the observed graph structure and represents meta-path instances as path-supported $2D$-cells, thereby preserving graph heterogeneity. Across the evaluated node-classification, graph-classification, and link-prediction benchmarks, H2S-NN achieved the strongest overall performance among the compared methods, supporting its effectiveness for heterogeneous heterophilic graph learning.

CCS Concepts

• **Computing methodologies** → **Neural networks**.

Keywords

heterogeneous graph neural networks, heterophily, sheaf neural networks, meta-path instances, diffusion

ACM Reference Format:

Anonymous Author(s). 2026. H2S-NN: Heterogeneous Heterophilic Sheaf Neural Network. In *Proceedings of ACM Conference (Conference acronym 'XX)*. ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Heterogeneous graphs are a common representation for modeling complex relational structures, such as citation networks [21] or social interactions [31]. Unlike homogeneous graphs, heterogeneous graphs contain multiple types of nodes and relations with distinct semantic roles [32]. Moreover, many graphs are also heterophilic, meaning that adjacent nodes may have different labels or attributes, making simple similarity-based neighbor aggregation unreliable and prone to oversmoothing. A graph can be both heterogeneous and heterophilic, resulting in a much more complex learning task called heterogeneous heterophilic graph learning, as shown in Figure 1. Furthermore, modeling graphs under the simplifying assumptions of homogeneity or homophily can obscure important structural and relational patterns. These simplifying assumptions can produce an incomplete representation of the original graph and consequently degrade predictive performance [20].

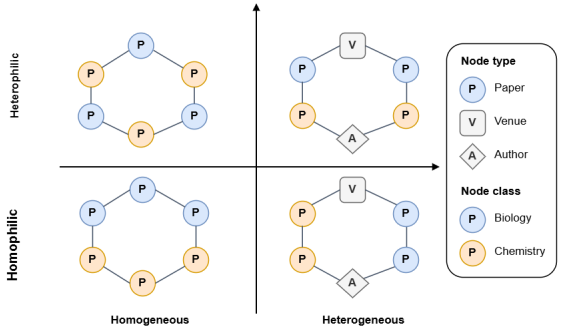


Figure 1: An example of a heterogeneous heterophilic graph. The graph contains multiple node and relation types, and adjacent nodes may have different labels or attributes.

Many existing methods address either heterogeneous graphs without explicitly considering heterophily [14, 17, 24, 38, 40, 43] or heterophilic graphs without modeling heterogeneous node and relation types [4, 11, 29, 42]. Recent methods have begun to address heterogeneous heterophilic graphs through meta-path-schema-based latent subgraphs [7, 12, 19, 22, 30, 37], graph rewiring [34], graph transformers [20], and sheaf-based mappings [5]. Another important concept is the use of meta-path schemas. These schemas can express information that is not contained in either node endpoint alone [33]. For instance, an `Author-Paper-Author` meta-path schema describes a co-authorship relation in citation networks, as shown in Figure 2. However, existing methods either ignore meta-path schemas completely, or use meta-path schemas as semantic views such as filtered edge sets or aggregation channels, rather than retaining each concrete path instance as part of the propagation structure. For example, an `A-P-A` meta-path schema can connect two authors through a shared paper, but after it is converted into an author-author view, the shared paper and the two authorship edges no longer directly shape the propagation operator. This node-endpoint-only representation is restrictive under heterophily because whether information should be reinforced or contrasted may depend on the relations and intermediate nodes along the path. Collapsing the path can therefore discard local relational structure and higher-order information carried by the concrete path instance.

To address this gap, we introduce the **Heterogeneous heterophilic sheaf neural network (H2S-NN)**, which represents concrete meta-path instances as path-supported $2D$-cells and incorporates them into a sheaf-Hodge propagation operator. Compatible node-to-edge and edge-to-face maps preserve the ordered relational context of each path while allowing relation-specific transformations of node and edge states. The H2S-NN also supports automatic

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, City, Country

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/MM
https://doi.org/XXXXXXX.XXXXXXX

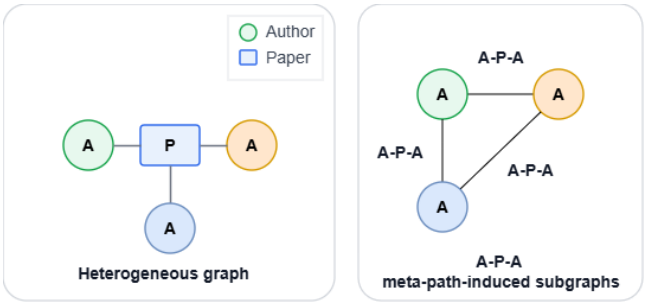


Figure 2: Illustration of meta-path-schema-induced graph transformation. Under the A-P-A meta-path schema, authors that share a paper in the original heterogeneous graph are connected in an author-level subgraph.

task-aligned selection of meta-paths. Our main contributions are summarized as follows:

(1) We propose a Heterogeneous heterophilic sheaf neural network (H2S-NN), which retains concrete path instances as path-supported $2D$-cells in an augmented path-supported 2D cellular complex, thereby preserving both the original graph structure and local relational structure under heterophily.
(2) We develop Adaptive Meta-Path Schema-to-Cell Searching and Selection (AMPSS), a component of H2S-NN that selects relevant meta-path schemas and identifies their task-relevant concrete instances.
(3) We develop Sheaf Covariance Pooling (SheafCovPool), a graph-level readout component of H2S-NN that produces permutation- and gauge-invariant graph representations for graph-level tasks.

As a result, H2S-NN provides an effective and generalizable model for heterogeneous heterophilic graphs.

2 Preliminaries

2.1 Heterogeneous graphs

A heterogeneous graph is a tuple $G = (V, E, \phi, \psi)$, where V and E are the node and edge sets, $\phi : V \to \mathcal{T}$ assigns each node a type, and $\psi : E \to \mathcal{R}$ assigns each edge a relation type. The realized type sets are $\mathcal{T} = \{\phi(v) : v \in V\}$ and $\mathcal{R} = \{\psi(e) : e \in E\}$. The graph is heterogeneous when it contains more than one node type or more than one relation type, i.e., $|\mathcal{T}| > 1$ or $|\mathcal{R}| > 1$; it reduces to a homogeneous graph when $|\mathcal{T}| = |\mathcal{R}| = 1$ [22].

2.2 Meta-Path Schemas and Path Instances

A meta-path schema of length ℓ is a typed relation sequence

$$\pi : t_0 \xrightarrow{r_1} t_1 \xrightarrow{r_2} \cdots \xrightarrow{r_\ell} t_\ell, \qquad t_i \in \mathcal{T},\ r_i \in \mathcal{R}, \tag{1}$$

which specifies a composite relation between its endpoint types [33]. For each relation r, let $A_r \in \{0,1\}^{|V|\times|V|}$ be its adjacency matrix, with $(A_r)_{uv} = 1$ when a relation of type r connects u to v. The schema induces the walk-count matrix

$$A_\pi = A_{r_1} A_{r_2} \cdots A_{r_\ell}, \tag{2}$$

where $(A_\pi)_{uv}$ counts the concrete walks from u to v that follow π. When $t_0 = t_\ell$, thresholding A_π produces a homogeneous endpoint graph on which Eq. (3) can be evaluated. For example, as shown in Figure 2, the paper-author-paper schema has $A_{\mathrm{PAP}} = A_{\mathrm{PA}}A_{\mathrm{AP}}$ and connects papers that share an author. A concrete path instance is an ordered sequence $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ satisfying $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Thus, π describes a type-level pattern, whereas f identifies one occurrence of that pattern in the observed graph.

2.3 Homophily and Heterophily

Homophily refers to the prevalence of edges between nodes that share the same label. For a graph $G' = (V', E')$ whose endpoint labels are comparable, its edge homophily is

$$H_{\mathrm{edge}}(G') = \frac{1}{|E'|} \sum_{(u,v)\in E'} \mathbf{1}[y_u = y_v], \tag{3}$$

where y_u is the label of node u and $\mathbf{1}[\cdot]$ is the indicator function [45]. Heterophily is the complementary tendency for adjacent nodes to have different labels. In a heterogeneous graph, labels are generally comparable only after a meta-path induces an endpoint graph whose nodes share the target type.

We therefore characterize heterogeneous node-classification graphs using metapath-based label heterophily (MLH) [20]. Let $\mathcal{P}_2(A)$ be the set of valid length-two meta-path schemas that start and end at the target node type A, and let G'_p denote the endpoint graph induced by $p \in \mathcal{P}_2(A)$. For each G'_p, define edge heterophily as

$$H_{\mathrm{het}}(G'_p) = 1 - H_{\mathrm{edge}}(G'_p). \tag{4}$$

We use the mean aggregation function over these induced graphs:

$$\mathrm{MLH}(G; A) = \frac{1}{|\mathcal{P}_2(A)|} \sum_{p\in\mathcal{P}_2(A)} H_{\mathrm{het}}(G'_p). \tag{5}$$

This mean gives each available semantic view equal weight, rather than allowing a dense induced graph to determine the dataset-level value. MLH lies in $[0, 1]$: values close to 0 indicate homophily, whereas values close to 1 indicate heterophily. For multilabel targets, two endpoint labels match only when their complete label vectors are equal.

2.4 Cellular Sheaves and Sheaf Neural Networks

Based on sheaf theory, a graph can be viewed as a one-dimensional cellular complex, with nodes as 0-cells and edges as 1-cells. More general cellular complexes may include higher-dimensional cells, such as $2D$-cells bounded by edges, but most sheaf neural networks on graphs operate on the node-edge complex. We write $c \trianglelefteq \tau$ when a lower-dimensional cell c is incident to a higher-dimensional cell τ.

A cellular sheaf $\mathcal{F}$ assigns each cell c a vector space, or *stalk*, $\mathcal{F}(c)$, and each incident pair $c \trianglelefteq \tau$ a linear restriction map $F_{c\trianglelefteq\tau} : \mathcal{F}(c) \to \mathcal{F}(\tau)$. On a graph $G = (V, E)$, node and edge assignments form the cochain spaces

$$C^0(G, \mathcal{F}) = \bigoplus_{v\in V} \mathcal{F}(v), \qquad C^1(G, \mathcal{F}) = \bigoplus_{e\in E} \mathcal{F}(e). \tag{6}$$

For an oriented edge $e = (u \to v)$, the first coboundary operator $\delta_0 : C^0(G, \mathcal{F}) \to C^1(G, \mathcal{F})$ compares endpoint signals after transport

into the edge stalk:

$$(\delta_0 x)_e = F_{v \trianglelefteq e} x_v - F_{u \trianglelefteq e} x_u. \tag{7}$$

The corresponding sheaf Laplacian is $L_0 = \delta_0^\top \delta_0$, with energy $x^\top L_0 x = \|\delta_0 x\|_2^2$. When all stalks are one-dimensional and all restriction maps are identities, L_0 recovers the ordinary graph Laplacian.

Sheaf neural networks can address oversmoothing [6], oversquashing [1], and heterophily by replacing direct feature averaging with diffusion through local stalks and learned restriction maps [4, 13], leading to numerous recent extensions and applications of sheaf neural networks on graphs and in other domains [3, 10, 16, 27–29]. HetSheaf further adapts sheaf maps to heterogeneous graphs [5]. However, these models define sheaf structure primarily over observed node-edge incidences. In contrast, H2S-NN retains selected concrete path instances as path-supported $2D$-cells in an augmented path-supported $2D$-cellular complex and constructs compatible edge-to-face maps, enabling diffusion over an explicit edge cochain.

3 Methodology

3.1 Problem Formulation

We consider supervised prediction on a directed heterogeneous graph in which neighboring nodes may have different labels or features. The graph is formulated as $G = (V, E, \phi, \psi, X^{\text{raw}})$, where $\phi : V \to \mathcal{T}$ assigns node types, $\psi : E \to \mathcal{R}$ assigns relation types, and $X^{\text{raw}} = \{x_v^{\text{raw}}\}_{v \in V}$ contains type-dependent node features with $x_v^{\text{raw}} \in \mathbb{R}^{d_{\phi(v)}}$.

The model is given meta-path schemas π, either supplied manually or selected by AMPSS. A schema $\pi = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$ specifies a valid typed relation sequence, while $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ denotes one concrete path instance satisfying $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Let $\mathcal{I}_\pi$ be the set of observed instances of schema π, and let $\mathcal{I}_\Sigma = \bigcup_{\pi \in \Sigma} \mathcal{I}_\pi$. Unlike methods that collapse a meta-path schema into an endpoint-level homogeneous graph, we represent each retained path instance as a path-supported 2-cell: a face-like higher-order structure whose boundary preserves the ordered path edges and includes a synthetic closure edge introduced below.

The learning problem is to construct a predictor

$$\hat{y}_o = g_\theta(o; G, \mathcal{I}_\Sigma),$$

for a task-specific supervised object o. For node classification, o is a target node; for link prediction, o is a candidate typed node pair; and for graph classification, o is an entire graph. Given labeled training objects $\mathcal{S}_{\text{train}}$, the parameters are learned by minimizing

$$\min_\theta \frac{1}{|\mathcal{S}_{\text{train}}|} \sum_{o \in \mathcal{S}_{\text{train}}} \mathcal{L}(g_\theta(o; G, \mathcal{I}_\Sigma), y_o)\,.$$

$\mathcal{L}$ is cross-entropy for single-label node and graph classification, and binary cross-entropy for multilabel node classification and link prediction. Thus, the central challenge is to learn type-aware and relation-aware propagation operators that can use selected concrete path instances as path-supported $2D$-cells on heterogeneous heterophilic graphs.

3.2 Overview

We introduce H2S-NN, a Heterogeneous heterophilic sheaf neural network, for supervised learning on heterogeneous heterophilic graphs. As shown in Figure 3, H2S-NN consists of four stages. First, *Input graph preprocessing* prepares typed graph features and relations. Second, AMPSS generates candidate meta-path schemas using bounded depth-first search, selects a task-relevant schema set (Stage I), and learns schema-conditioned weights for its retained concrete instances (Stage II). Third, the *Sheaf-Hodge Encoding* stage incorporates the selected concrete path instances to learn node-to-edge and path-supported edge-to-face geometry. Finally, the *Sheaf-Hodge Diffusion* stage evolves the node and edge cochains to obtain their final representations, with the proposed SheafCovPool provided as the graph-level readout. Together, these stages allow H2S-NN to retain the local relational structure of selected concrete path instances while learning from heterogeneous graphs under heterophily.

3.3 Input Graph Preprocessing: Initialization of Node Stalks and Sheaf Restrictions

H2S-NN first maps type-dependent node features into a shared d-dimensional local feature space (stalk space). This makes representations from different node types compatible with relation-specific restrictions, which transform the endpoint representations into a common edge-stalk space before their disagreement is computed. For each node v, we compute

$$h_v^{(0)} = \text{Dropout}\big[\text{GELU}\big(\text{LN}\left(W_{\phi(v)} x_v^{\text{raw}} + b_{\phi(v)}\right) + q_{\phi(v)}\big)\big] \in \mathbb{R}^d, \tag{8}$$

where W_t and b_t are type-specific parameters and q_t is a learned type embedding. When raw attributes are unavailable, we use a trainable node embedding $e_v \in \mathbb{R}^d$ directly as the initial representation $h_v^{(0)}$. We initialize the node cochain as $x^{(0)} = h^{(0)}$ and associate each relation r with an embedding $\zeta_r \in \mathbb{R}^{d_r}$.

For an oriented typed edge $e = (u \xrightarrow{r} v)$, two relation-conditioned networks infer the endpoint restrictions $F_{u \trianglelefteq e}, F_{v \trianglelefteq e} \in \mathbb{R}^{d \times d}$ from $h_u^{(0)}$, $h_v^{(0)}$, and ζ_r. The source and target restrictions use the same parameterization:

$$F_{z \trianglelefteq e} = \lambda_{z,e} Q_{z,e}, \qquad Q_{z,e}^\top Q_{z,e} = I, \qquad \lambda_{z,e} > 0, \quad z \in \{u, v\}. \tag{9}$$

Here, the orthogonal factor aligns endpoint coordinates, while the positive scale modulates the magnitude of the relation-aware disagreement computed by δ_0, rather than comparing raw features directly.

We initialize the collection of edge representations (edge cochain) independently from this disagreement:

$$s_e^{(0)} = \text{MLP}_{\text{init}}\Big(h_u^{(0)} \,\|\, h_v^{(0)} \,\|\, \zeta_r \,\|\, (\delta_0 h^{(0)})_e\Big)\,. \tag{10}$$

Thus, $s^{(0)}$ is not restricted to $\text{im}\,\delta_0$, allowing the subsequent node-mediated and face-induced edge interactions to act on edge-level information that is not induced solely by node representations. After the selected path instances introduce closure 1-cells, the same coboundary and initialization rules are evaluated over the extended edge set.

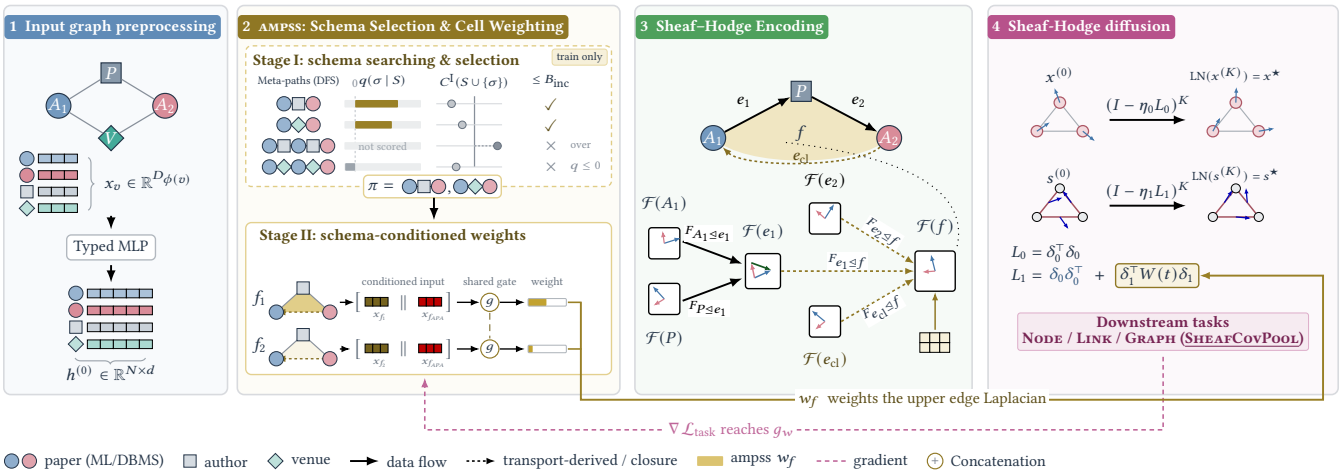


Figure 3: Overview of H2S-NN on a citation graph. (1) Type-specific encoders map heterogeneous node features to a common d-dimensional stalk representation. (2) AMPSS uses DFS to enumerate meta-path candidates and an auxiliary linear selection model fitted only on the training data to select meta-path schemas in Stage I. In Stage II, it learns schema-conditioned weights for the retained concrete path instances. (3) Each selected simple path instance is completed by a synthetic closure 1-cell to form a path-supported $2D$-cell. Learned node-to-edge maps on observed edges define path transports, which construct the closure restrictions and compatible edge-to-face maps in the corresponding face-stalk frame. (4) Node and edge cochains undergo K parallel Hodge-diffusion steps before task-specific prediction. The learned instance weights modulate the face-induced edge interactions, allowing the supervised task loss to update AMPSS jointly with the predictor.

3.4 AMPSS: Adaptive meta-path schema-to-cell searching and selection

AMPSS is a supporting mechanism for constructing a task-specific set of meta-path instances. AMPSS has two stages. Stage I generates candidate meta-path schemas through bounded depth-first search and selects a task-aligned subset of meta-path schemas from the training data. Stage II assigns schema-conditioned weights to the retained path instances, controlling their contribution to the face-induced edge operator. Thus, Stage I selects typed path patterns, whereas Stage II adapts the influence of individual instances to the supervised task.

3.4.1 Stage I: task-aligned schema searching and selection. Stage I identifies a compact set of task-relevant meta-path schemas. We construct a type graph from the relation types available during Stage I and use bounded depth-first search to generate a candidate pool Π of schemas $\pi = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$, with lengths between $\ell_{\min}$ and $\ell_{\max}$. Candidates with no feasible simple path instances are discarded. An edge-face incidence is one occurrence of a boundary edge in a path-supported 2-cell.

For a trial schema set $S \subseteq \Pi$, deterministic instance and incidence quotas define the retained instances $\mathcal{I}^q_\pi(S)$. These instances provide features derived from the retained path instances of each schema for an auxiliary linear selection model and determine the structural cost. Stage I greedily selects schemas subject to a schema limit and instance and incidence budgets.

The structural cost is the number of retained edge-face incidences:

$$C^{\mathrm{I}}(S) = \sum_{\pi \in S} \sum_{f \in \mathcal{I}^q_\pi(S)} (\ell_f + 1), \tag{11}$$

where each instance contributes its ℓ_f path edges and one closure edge. Thus, the incidence budget directly bounds the support of the resulting face-induced edge operator.

The auxiliary linear selection model is fitted on an inner-training partition and evaluated on a disjoint held-out selection partition. The gain from adding a candidate schema π is

$$q(\pi \mid S) = \mathcal{L}_{\mathrm{sel}}(S) - \mathcal{L}_{\mathrm{sel}}(S \cup \{\pi\}), \qquad \pi \in \mathcal{A}(S),$$
$$\mathcal{A}(S) = \left\{ \pi \in \Pi \setminus S \;\middle|\; \begin{array}{l} |S \cup \{\pi\}| \le K_\Sigma, \\ C^{\mathrm{I}}(S \cup \{\pi\}) \le B_{\mathrm{inc}} \end{array} \right\}. \tag{12}$$

Starting from $S = \varnothing$, Stage I adds the feasible schema that yields the largest positive reduction in held-out selection loss. The selection model is refitted for each trial set using labels from the outer training split only. For link prediction, inner-selection positive links and their reverse counterparts are excluded from the graph used for selection. After selection, the retained instances are constructed on the full outer-training graph. Because schema selection is discrete, gradients from the H2S-NN task loss do not update Stage I. Thus, Stage I provides a task-aligned schema set for Stage II.

3.4.2 Stage II: schema-conditioned instance weighting. For the schema pool selected in Stage I, Stage II assigns a positive weight to each retained concrete path instance. Instances sharing the same meta-path schema can encode different task-relevant information. Let ξ_f denote the representation of a retained instance f produced by the ordered path encoder before constructing its edge-to-face map A_f, and let $e_{\pi(f)}$ denote the embedding of the schema of f. We define

$$w_f(\tau) = (1 - r(\tau)) + 2r(\tau)\ \mathrm{sigmoid}\big(g_w(\xi_f \,\|\, e_{\pi(f)})\big), \tag{13}$$

where $r(\tau) \in [0,1]$ is a warm-up schedule that gradually changes the weights from uniform to learned values. The output layer of g_w is initialized at zero; hence, $w_f(\tau) = 1$ at initialization, and learning starts from the unweighted face-induced edge interactions. The schema embedding enables the weighting function to distinguish instances from different schemas, such as PAP and PSP, whereas ξ_f distinguishes instances within the same schema. The weights are learned solely through the supervised task loss.

Let $W(\tau) = \mathrm{diag}(w_f(\tau))$. Stage II weights the face-induced component of the edge operator as $\delta_1^\top W(\tau)\delta_1$. Because $w_f(\tau) > 0$, this component remains positive semidefinite. The weights therefore modulate only the contribution of retained path-supported $2D$-cells. Closure edges remain part of the extended edge domain and continue to contribute to δ_0, L_0, the node-mediated component of the edge operator, the weighted face boundary in δ_1, and the edge readout.

Thus, Stage II provides task-driven, instance-level modulation of the selected path-supported $2D$-cells while preserving the extended edge structure.

3.5 Sheaf-Hodge Encoding

Because nodes, edges, and selected concrete path instances must be incorporated into the sheaf geometry before they can influence diffusion, we introduce a Sheaf-Hodge encoding to address this requirement. It augments the observed graph with path-supported $2D$-cells and defines compatible node-to-edge and edge-to-face restriction maps, preserving the ordered structure of each selected path while enabling node and edge cochains to exchange information through observed relations and retained higher-order supports.

3.5.1 Augmented Path-Supported 2-Cell Construction. The Sheaf-Hodge encoder augments the observed graph with retained concrete path instances. In AMPSS mode, Stage I selects the corresponding meta-path schemas and determines the retained instances through its quotas. Rather than replacing an instance with an endpoint-only relation, H2S-NN retains the ordered path and augments it with a synthetic closure 1-cell

$$f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$$

as a path-supported $2D$-cell whose boundary preserves the ordered path edges.

Because this path is open, we introduce a synthetic closure 1-cell, $e_{\mathrm{cl}} = (v_\ell \to v_0)$, to form the closed boundary

$$\partial f = (e_1, \ldots, e_\ell, e_{\mathrm{cl}}). \tag{14}$$

The closure 1-cell is computationally active in the extended edge domain. Its restriction maps are determined by path transport rather than learned independently.

Let E_{cl} contain one closure 1-cell for each retained instance, and let $E_{\mathrm{ext}} = E \sqcup E_{\mathrm{cl}}$. The retained instance set is

$$\mathcal{I}_{\mathrm{ret}} = \bigcup_{\pi \in S} \mathcal{I}_\pi^q(S),$$

where S denotes the selected schema set. With d-dimensional node, edge, and face stalks, the cochain spaces are

$$C^0(\mathcal{F}) = \bigoplus_{v \in V} \mathcal{F}(v),\ C^1(\mathcal{F}) = \bigoplus_{e \in E_{\mathrm{ext}}} \mathcal{F}(e),\ C^2(\mathcal{F}) = \bigoplus_{f \in \mathcal{I}_{\mathrm{ret}}} \mathcal{F}(f). \tag{15}$$

After the closure restrictions are defined by the path transport described below, the first coboundary in Equation (7) is evaluated over E_{ext}. Hence, closure 1-cells are included in the extended edge domain and participate in the initial edge cochain and in both L_0 and L_1. We next define the edge-to-face restriction maps for these path-supported $2D$-cells in the following section.

3.5.2 Path-Supported Edge-to-Face Maps. The edge-face construction turns each selected concrete path instance into a path-supported $2D$-cellular graph. For a selected $P{-}A{-}P$ instance, for example, the two paper-author edges remain the boundary edges of one path-supported $2D$-cell. The intermediate author remains part of the operator through the ordered path boundary, while the synthetic closure edge adds a path-transport-constrained coupling between the endpoints. This lets the face-induced edge interactions couple disagreements along the whole paper-author-paper pattern.

For a general instance $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$, H2S-NN first predicts one free map $A_f \in \mathbb{R}^{d \times d}$ from the ordered path. Each face group contains path instances of a single schema, so all instances in the group have the same length ℓ and relation sequence $(r_1, \ldots, r_\ell)$. The default face encoder concatenates the node states in path order together with the schema's relation embeddings:

$$A_f = \mathrm{MLP}_\ell\Big(h_{v_0}^{(0)} \,\|\, \cdots \,\|\, h_{v_\ell}^{(0)} \,\|\, \zeta_{r_1} \,\|\, \cdots \,\|\, \zeta_{r_\ell}\Big) \in \mathbb{R}^{d \times d}. \tag{16}$$

Here, MLP_ℓ is a path-length-specific MLP that maps the ordered path representation to the entries of A_f. The resulting map A_f defines the local coordinate system of the face stalk for instance f. We construct the edge-to-face maps by transporting A_f along the ordered path through the node-to-edge restriction maps. This construction ensures that the face-induced component measures edge-level inconsistency around a selected path-supported 2-cell, rather than creating an artificial face signal from an edge cochain induced solely by node representations. Starting from the base vertex v_0, the transport is

$$\begin{aligned} T_{v_0} &= A_f, \qquad T_{v_i} = T_{v_{i-1}} F_{u \trianglelefteq e_i}^{-1} F_{v \trianglelefteq e_i}, \\ F_{e_i \trianglelefteq f} &= T_{v_{i-1}} F_{u \trianglelefteq e_i}^{-1}, \end{aligned} \tag{17}$$

where $u = v_{i-1}$ and $v = v_i$.

The closure edge uses the same path transport:

$$F_{v_0 \trianglelefteq e_{\mathrm{cl}}} = I, \qquad F_{v_\ell \trianglelefteq e_{\mathrm{cl}}} = T_{v_0}^{-1} T_{v_\ell}, \qquad F_{e_{\mathrm{cl}} \trianglelefteq f} = A_f. \tag{18}$$

The middle map is the transformation accumulated along the path $\prod_i F_{u \trianglelefteq e_i}^{-1} F_{v \trianglelefteq e_i}$. Hence, the closure edge introduces no additional learned restriction map.

Let $\eta(e, f) \in \{-1, +1\}$ denote the incidence orientation. The weighted second coboundary is

$$(\delta_{1,w} s)_f = \sqrt{w_f} \sum_{e \in \partial f} \eta(e, f) F_{e \trianglelefteq f} s_e. \tag{19}$$

Writing $B_f s = \sum_{e \in \partial f} \eta(e, f) F_{e \trianglelefteq f} s_e$, the corresponding face-induced contribution is

$$\delta_{1,w}^\top \delta_{1,w} = \sum_{f \in \mathcal{I}_{\mathrm{ret}}} w_f B_f^\top B_f. \tag{20}$$

Proposition 1 (Exact cochain condition). *For every path instance constructed by H2S-NN whose node sequence is a simple path and whose boundary follows the default forward orientation, the*

maps in Eqs. (17) - (18) *satisfy* $\delta_{1,w}\delta_0 = 0$ *identically for every node cochain and every value of the learned parameters.*

Proof. By construction $F_{e_i \trianglelefteq f} F_{u \trianglelefteq e_i} = T_{v_{i-1}}$ and $F_{e_i \trianglelefteq f} F_{v \trianglelefteq e_i} = T_{v_i}$. The i-th path term of $\delta_{1,w}\delta_0 x$ is therefore $T_{v_i} x_{v_i} - T_{v_{i-1}} x_{v_{i-1}}$, and summing over the path telescopes to $T_{v_\ell} x_{v_\ell} - T_{v_0} x_{v_0}$. The closure maps give $F_{e_{\mathrm{cl}} \trianglelefteq f} F_{v_0 \trianglelefteq e_{\mathrm{cl}}} = T_{v_0}$ and $F_{e_{\mathrm{cl}} \trianglelefteq f} F_{v_\ell \trianglelefteq e_{\mathrm{cl}}} = T_{v_\ell}$, so the closure term contributes $T_{v_0} x_{v_0} - T_{v_\ell} x_{v_\ell}$. The total sum is zero. The simple-path condition is required because the transport assigns one map to each node position; a revisited node, including a coincident endpoint pair, would require two transported values for the same node. □

The chain condition in Proposition 1 is scoped to the implemented simple-path, transport-composed faces and their aligned closure edges. Within that scope it is not optimized as an additional penalty but enforced by construction. Its role is to keep the interpretation of the Hodge decomposition clean: edge signals in $\operatorname{im} \delta_0$ come from edge signals induced by node representations, while the face-induced component acts on edge-level residuals around selected path-supported cycles.

3.6 Sheaf-Hodge Diffusion

After the restriction maps have been constructed, H2S-NN diffuses node and edge cochains using one node operator and one edge operator:

$$L_0 x = \delta_0^{\mathsf{T}} \delta_0 x, \tag{21}$$

$$L_1 s = \underbrace{\delta_0 \delta_0^{\mathsf{T}} s}_{\text{node-mediated component}} + \underbrace{\delta_{1,w}^{\mathsf{T}} \delta_{1,w} s}_{\text{face-induced component}}. \tag{22}$$

The node operator L_0 compares transported endpoint signals. The edge diffusion operator L_1 contains a node-mediated component and a face-induced component. The node-mediated component of L_1 relates edge signals through shared incident nodes, whereas the face-induced component couples boundary edges within selected path-supported $2D$-cells. In Hodge-theoretic terminology, these are the lower and upper components of the edge operator, which represent node-mediated and face-induced edge interactions, respectively. Because each closure 1-cell is part of the extended edge set, it participates in both components without requiring a separate node-side correction.

Starting from $x^{(0)} = h^{(0)}$ and the independently initialized edge cochain $s^{(0)}$, K explicit Euler steps give

$$\begin{aligned} x^{(k+1)} &= x^{(k)} - \alpha_0 L_0 x^{(k)}, \\ s^{(k+1)} &= s^{(k)} - \alpha_1 L_1 s^{(k)}, \qquad k = 0, \dots, K-1. \end{aligned} \tag{23}$$

The positive step sizes may be learned. Node and edge cochains are updated by separate L_0 and L_1 operators. Although updated node and edge states are not exchanged between diffusion steps, both operators share the learned restriction geometry and initial node representations. Layer normalization then gives the terminal states $x^\star$ and $s^\star$, which are passed to the task-specific readout.

The Hodge operators are evaluated through local restriction maps and indexed aggregation without explicitly forming global operators. The implementation batches local restriction maps and aggregates contributions with indexed sums. This applies δ_0, δ_0^{T}, $\delta_{1,w}$, and $\delta_{1,w}^{\mathsf{T}}$ directly to node, edge, and face tensors.

Three implementation boundaries are important. First, $\delta_{1,w}$ uses the complete boundary cycle, consisting of the ℓ path edges and the closure 1-cell. Second, incidence signs are represented explicitly. The paths produced by the current enumeration use +1 signs because the enumeration follows stored edge directions and orients the closure edge along the cycle. Third, the exact transport argument assumes simple paths, so instances that revisit a node, including coincident endpoints, are excluded during enumeration. Under these constraints, the chain condition is a property of the constructed operator, not a penalty added to the training objective.

The shared backbone produces final node states $x^\star$ and edge states $s^\star$. For node classification, the target-node state is combined with pooled incident-edge information. For link prediction, endpoint representations are formed from node and incident-edge states before applying a pair decoder. For graph classification, SheafCovPool aggregates node and edge representations into a graph-level vector, as described next.

3.7 SheafCovPool

Graph classification requires one representation for the whole graph, yet node and edge states are expressed in different local coordinate systems. Averaging incident edge coordinates directly would therefore mix vectors from different spaces. Before pooling, SheafCovPool projects each incident edge state into the corresponding node coordinate space using the transpose of its restriction map:

$$\widetilde{s}_v^\star = \frac{1}{\deg_{\mathrm{ext}}(v)} \left(\sum_{e:\mathrm{src}(e)=v} F_{\mathrm{src},e}^{\mathsf{T}} s_e^\star + \sum_{e:\mathrm{tgt}(e)=v} F_{\mathrm{tgt},e}^{\mathsf{T}} s_e^\star \right). \tag{24}$$

Here $F_{\mathrm{src},e}$ and $F_{\mathrm{tgt},e}$ denote the restriction maps from the source and target nodes of e, respectively, into the edge stalk. $\deg_{\mathrm{ext}}(v)$ is the number of incidences at v over observed and closure edges; an isolated node receives the zero vector. Both endpoints contribute positively because this quantity summarizes incident context rather than the orientation-sensitive divergence $\delta_0^{\mathsf{T}} s^\star$. The sums range over the observed and closure edges, so both types of 1-cell contribute.

The readout then forms three complementary views at each node and edge:

$$\begin{aligned} Z_v^{(0)} &= \begin{bmatrix} x_v^\star & h_v^{(0)} & \widetilde{s}_v^\star \end{bmatrix} \in \mathbb{R}^{d \times 3}, \\ Z_e^{(1)} &= \begin{bmatrix} (\delta_0 x^\star)_e & (\delta_0 h^{(0)})_e & s_e^\star \end{bmatrix} \in \mathbb{R}^{d \times 3}, \\ G_i^{(q)} &= (Z_i^{(q)})^{\mathsf{T}} Z_i^{(q)} \in \mathbb{R}^{3 \times 3}, \qquad q \in \{0, 1\}. \end{aligned} \tag{25}$$

This matrix records the energies and pairwise alignments of the three views while removing their common orthogonal orientation; we call it a Gram matrix. For an ordered view stack, equality of Gram matrices characterizes equality up to an orthogonal change of basis.

To see why the transported node view has this property, let $Q_v, R_e \in O(d)$ be independent changes of basis in the node and edge stalks. Under the consistent transformation

$$\begin{aligned} F'_{v \trianglelefteq e} &= R_e F_{v \trianglelefteq e} Q_v^{\mathsf{T}}, \qquad s_e^{\star\prime} = R_e s_e^\star, \\ (F'_{v \trianglelefteq e})^{\mathsf{T}} s_e^{\star\prime} &= Q_v F_{v \trianglelefteq e}^{\mathsf{T}} s_e^\star. \end{aligned} \tag{26}$$

Hence $Z_v^{(0)}$ transforms by Q_v, while $Z_e^{(1)}$ transforms by R_e, and both Gram descriptors remain unchanged. This argument requires orthogonal basis changes; the learned restrictions themselves may be scaled orthogonal maps.

Nodes and edges serve different roles, so separate networks project their Gram matrices. Within graph g, attention over these invariant descriptors yields first- and second-order summaries:

$$\begin{aligned} u_i^{(q)} &= \rho_q(\operatorname{vec} G_i^{(q)}) \in \mathbb{R}^F, \\ a_i^{(q)} &= \frac{\exp(g_q(u_i^{(q)}))}{\sum_{j\in C_q(g)} \exp(g_q(u_j^{(q)}))}, \\ \mu_q(g) &= \sum_{i\in C_q(g)} a_i^{(q)} u_i^{(q)}, \\ \Sigma_q(g) &= \sum_{i\in C_q(g)} a_i^{(q)} (u_i^{(q)} - \mu_q(g))(u_i^{(q)} - \mu_q(g))^\top. \end{aligned} \tag{27}$$

Here $q \in \{0,1\}$, $C_0(g)$ is the node set, and $C_1(g)$ contains the observed and closure edges. The mean describes the evidence emphasized by attention, whereas the covariance records how that evidence varies across the graph. The final graph representation is

$$\gamma_g = \left[\mu_0(g) \,\|\, \operatorname{triu}(\Sigma_0(g)) \,\|\, \mu_1(g) \,\|\, \operatorname{triu}(\Sigma_1(g))\right]. \tag{28}$$

The order of the raw Gram matrix is fixed at three because it is determined by the three semantic views in each stack, rather than by a freely chosen hidden width. Changing that order would require adding or removing stalk-valued views and would therefore define a different readout. We instead control readout capacity through the post-Gram projection width F. The two means and two upper-triangular covariance vectors give $\gamma_g \in \mathbb{R}^{F^2+3F}$. Because every learned score acts on an invariant Gram matrix and every aggregation is symmetric, γ_g is invariant to node and edge ordering. Given consistently transformed states and restriction maps, it is also invariant to independent orthogonal changes of basis in the node and edge stalks.

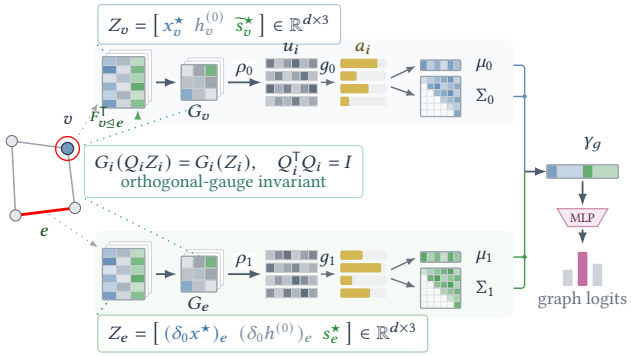


Figure 4: SheafCovPool graph readout. Incident edge states are first expressed in their endpoint node stalks. Node and edge view stacks then become gauge-invariant Gram matrices G_i; attention-weighted means and covariances form the graph representation γ_g.

4 Experiments

4.1 Experiment Settings

To assess the generalizability of H2S-NN across heterogeneous graph learning settings, we evaluated it on three tasks: node classification, link prediction, and graph classification.

Datasets. For node classification, we used three heterogeneous graph benchmarks: DBLP and IMDB from HGB [23] and ACM [39]. We also calcuated each metapath-based label heterophily (MLH) for each dataset. For graph classification, we selected four datasets from TUDataset [26]: two biological datasets, PROTEINS and ENZYMES, and two chemical-compound datasets, MUTAG and NCI1. To make the graph heterogeneous, we interpreted the node and edge labels provided by TUDataset as node and relation types, respectively. For link prediction, we used four HGB benchmarks [23]: three user-item interaction networks, LastFM, YouTube, and Amazon, and one biological knowledge graph, PubMed. Dataset statistics are reported in Tables 1, 2, and 3.

Baselines. For node classification, we compared H2S-NN with baselines from four model categories. The homogeneous homophilic baselines comprised GCN [18], GAT [36], and ChebNetII [15], while the homogeneous heterophilic baselines included NSD [4] and CSNN [29]. For heterogeneous graphs, we considered the homophilic models HAN [39], SeHGNN [41], HiNormer [24], and PSHGCN [14], as well as the heterophilic models H2G-former [20], Hetero^{2}Net [19], H2SGNN [22], HetSheaf [5], and LatGRL [30]. For graph classification and link prediction, we used a subset of these baselines while retaining representatives from all four categories. Specifically, we included GCN [18] and GAT [36] as homogeneous homophilic models; NSD [4] and CSNN [29] as homogeneous heterophilic models; HAN [39] and PSHGCN [14] as heterogeneous homophilic models; and H2G-former [20], Hetero^{2}Net [19], H2SGNN [22], HetSheaf [5], and LatGRL [30] as heterogeneous heterophilic models.

Evaluation Settings. For node classification, we followed the HGB benchmark protocol [23], using transductive training/validation/test splits of 24%/6%/70% and reporting Macro-F1 and Micro-F1. For graph classification, unlike prior work [2, 44], we performed 10-fold cross-validation over five random seeds and reported the mean and standard deviation of Macro-F1 and Micro-F1 to evaluate model performance unbiasedly without overestimating model performance. For the multi-class ENZYMES dataset, we adopted a one-vs-rest strategy. For link prediction, we followed the HGB protocol [23], using transductive splits of 81%/9%/10% and reporting AUPR and AUROC. Negative edges were sampled from the 2-hop neighborhood at a 1:1 ratio to positive edges. All models were trained with early stopping using a patience of 100 epochs. Hyperparameters were selected through per-dataset sweeps, with the complete configurations provided in Appendix B. We group the baselines according to graph type and homophily assumption as Hom./hom., Hom./het., Het./hom., and Het./het., where the first term indicates whether the graph is modeled as homogeneous or heterogeneous and the second indicates whether the method is designed for homophilic or heterophilic settings. Results are averaged over five random seeds and reported with standard deviations. The code is available at placeholder_for_code_repository.com.

4.2 Results

For the node-classification tasks, the results in Table 1 show that H2S-NN achieved the best performance on all three benchmark datasets, outperforming every compared baseline. This demonstrates the effectiveness of H2S-NN in capturing complex heterogeneous graph structures and relationships, leading to improved node classification performance.

Moreover, for the graph classification tasks, table 2 shows that H2S-NN tied the best reported MUTAG score and achieved the highest mean Macro-F1 and Micro-F1 on PROTEINS, ENZYMES, and NCI1 among the compared baselines. This suggests that treating the typed node and edge labels as heterogeneous structure, rather than as plain homogeneous graphs with categorical attributes, remains useful even outside the citation- and social-network domains where heterogeneity is usually studied.

The link-prediction results in Table 3 show that H2S-NN achieves the best performance on all four link-prediction datasets and all eight reported metrics. This advantage is consistent across multi-node-type graphs (LastFM and PubMed) and single-node-type, multi-relation graphs (Amazon and YouTube). These results indicate that H2S-NN supports link prediction on both multi-node-type graphs and single-node-type, multi-relation graphs.

5 Discussion

5.1 Ablation Study: Contribution of AMPSS

To evaluate AMPSS, we compared the full H2S-NN on DBLP node classification with variants using no path-supported $2D$-cells, predefined meta-path schemas, AMPSS with Stage-I selection only, and Stage-II weighting without the instance gate. As shown in Table 4, removing path-supported $2D$-cells decreased Macro-F1/Micro-F1, confirming that path-supported $2D$-cells contribute to node classification. Predefined meta-path schemas recovered most of this loss, while the full AMPSS model performed best. These results suggest that automatic meta-path schema selection can improve performance, but schema-level selection alone is insufficient; instance-level weighting helps filter noisy or uninformative path realizations.

Table 4: Ablation Study of AMPSS on DBLP node classification.

Configuration	Macro-F1	Micro-F1
No path-supported $2D$-cells	93.85	93.92
Predefined meta-path schemas	95.75	96.01
AMPSS, stage I only	95.88	96.02
AMPSS, schema weight only	95.91	96.08
AMPSS, instance weight only	95.95	96.13
AMPSS	**96.01**	**96.31**

5.2 Ablation Study: Contribution of SheafCovPool

To evaluate SheafCovPool, we compared the full H2S-NN on ENZYMES graph classification with sum, mean, and max pooling, SheafPool [5], and SheafCovPool. As shown in Table 5, SheafPool improved Macro-F1/Micro-F1 to 64.05/64.96, indicating that, under this protocol, a gauge-invariant readout retained information not captured by the three plain aggregation baselines. SheafCovPool performed best because SheafCovPool pools both node and edge Gram matrices and retains attention-weighted means together with across-node and across-edge covariances. These results suggest that gauge invariance is useful but not sufficient, and that retaining second-order statistics of the node and edge cochains is important for graph-level prediction.

Table 5: Ablation study of graph-level readouts on ENZYMES graph classification.

Readout	Macro-F1	Micro-F1
Sum + MLP	61.25	61.79
Mean + MLP	61.42	62.09
Max + MLP	61.38	61.62
SheafPool	64.05	64.96
SheafCovPool	**64.52**	**65.10**

6 Conclusion

To address heterogeneous graph learning under heterophily, we proposed Heterogeneous Heterophilic Sheaf Neural Network (H2S-NN), a graph neural network for supervised learning tasks. The model preserves concrete meta-path instances as path-supported $2D$-cells instead of reducing them to endpoint-level relations. It incorporates the ordered boundaries of these path-supported structures into the propagation operator through learned restriction maps and sheaf-Hodge diffusion, allowing information to flow through the node and edge cochains while retaining the relational context of each path. Across the evaluated benchmarks, H2S-NN achieved the strongest performance among the compared methods on node classification, graph classification, and link prediction. Overall, H2S-NN demonstrates that combining the preservation of meta-path and graph structures with sheaf-based propagation provides an effective and generalizable framework for heterogeneous heterophilic graph learning.

References

[1] Uri Alon and Eran Yahav. 2021. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *International Conference on Learning Representations*. https://openreview.net/forum?id=i80OPhOCVH2

[2] Alireza Amouzad, Zahra Dehghanian, Saeed Saravani, Maryam Amirmazlaghani, and Behnam Roshanfekr. 2024. Graph isomorphism u-net. *Expert Systems with Applications* 236 (2024), 121280.

[3] Federico Barbero, Cristian Bodnar, Haitz Sáez de Ocáriz Borde, Michael Bronstein, Petar Veličković, and Pietro Liò. 2022. Sheaf Neural Networks with Connection Laplacians. In *Proceedings of Topological, Algebraic, and Geometric Learning Workshops 2022 (Proceedings of Machine Learning Research, Vol. 196)*, Alexander Cloninger, Timothy Doster, Tegan Emerson, Manohar Kaul, Ira Ktena, Henry Kvinge, Nina Miolane, Bastian Rieck, Sarah Tymochko, and Guy Wolf (Eds.). PMLR, 28–36. https://proceedings.mlr.press/v196/barbero22a.html

[4] Cristian Bodnar, Francesco Di Giovanni, Benjamin Chamberlain, Pietro Lio, and Michael Bronstein. 2022. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *Advances in Neural Information Processing Systems* 35 (2022), 18527–18541.

[5] Luke Braithwaite, Alessio Borgi, Gabriele Onorato, Kristjan Tarantelli, Francesco Restuccia, Fabrizio Silvestri, and Pietro Liò. 2024. Heterogeneous sheaf neural networks. *arXiv preprint arXiv:2409.08036* (2024).

[6] Chen Cai and Yusu Wang. 2020. A note on over-smoothing for graph neural networks. *arXiv preprint arXiv:2006.13318* (2020).

[7] Qin Chen and Guojie Song. 2025. Adaptive heterogeneous graph neural networks: Bridging heterophily and heterogeneity. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 312–321.

Table 1: Performance on node classification benchmarks measured by Macro-F1 and Micro-F1. Best values are bold. The numbers in parentheses of datasets indicate the homophily ratio.

		DBLP (0.1277)		ACM (0.2639)		IMDB (0.7634)	
Group	**Model**	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
Hom./hom.	GCN	90.84 ± 0.32	91.47 ± 0.34	92.17 ± 0.24	92.12 ± 0.23	57.88 ± 1.18	64.82 ± 0.64
	GAT	93.83 ± 0.27	93.39 ± 0.30	92.26 ± 0.94	92.19 ± 0.93	58.94 ± 1.35	64.86 ± 0.43
	ChebNetII	92.05 ± 0.53	92.97 ± 0.48	92.45 ± 0.37	92.33 ± 0.38	62.54 ± 1.29	68.33 ± 0.92
Hom./het.	NSD	92.96 ± 0.21	93.42 ± 0.37	92.33 ± 0.78	92.05 ± 0.69	61.61 ± 0.26	65.23 ± 0.51
	CSNN	93.73 ± 0.95	92.12 ± 0.19	93.13 ± 0.94	92.81 ± 0.83	62.92 ± 0.34	62.59 ± 0.22
Het./hom.	HAN	91.67 ± 0.49	92.05 ± 0.62	90.89 ± 0.43	90.79 ± 0.43	57.74 ± 0.96	64.63 ± 0.58
	SeHGNN	95.06 ± 0.17	95.42 ± 0.17	94.05 ± 0.35	93.98 ± 0.36	67.11 ± 0.25	69.17 ± 0.43
	HiNormer	94.94 ± 0.21	94.57 ± 0.23	91.87 ± 0.26	89.64 ± 0.39	67.83 ± 0.34	64.65 ± 0.53
	PSHGCN	95.27 ± 0.13	95.61 ± 0.12	94.35 ± 0.23	94.27 ± 0.23	72.33 ± 0.57	74.46 ± 0.32
Het./het.	H2G-former	95.35 ± 0.81	95.00 ± 0.77	93.99 ± 0.35	94.13 ± 0.87	59.41 ± 0.22	59.63 ± 0.35
	Hetero^{2}Net	94.03 ± 0.35	94.46 ± 0.37	94.01 ± 0.54	93.91 ± 0.61	72.33 ± 0.57	74.46 ± 0.32
	H2SGNN	95.19 ± 0.11	95.56 ± 0.11	94.47 ± 0.25	94.38 ± 0.26	73.04 ± 0.65	75.46 ± 0.43
	HetSheaf	93.87 ± 0.00	93.77 ± 0.00	95.09 ± 0.00	94.94 ± 0.00	64.65 ± 0.00	67.18 ± 0.00
	LatGRL	94.32 ± 0.44	94.51 ± 0.42	93.47 ± 0.71	92.29 ± 0.43	58.27 ± 0.46	58.81 ± 0.65
Ours	**H2S-NN**	**96.01 ± 0.58**	**96.31 ± 0.40**	**95.91 ± 1.15**	**95.72 ± 0.68**	**75.05 ± 0.22**	**76.68 ± 0.19**

Table 2: Performance on graph classification benchmarks measured by Macro-F1 and Micro-F1. Best values are bold.

		MUTAG		PROTEINS		ENZYMES		NCI1	
Group	**Model**	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
Hom./hom.	GCN	89.58 ± 0.63	90.00 ± 0.38	75.92 ± 0.55	79.83 ± 0.49	12.75 ± 0.01	28.33 ± 0.03	64.17 ± 0.29	64.96 ± 0.38
	GAT	89.58 ± 0.06	90.00 ± 0.96	75.27 ± 0.59	78.38 ± 0.71	14.03 ± 0.05	16.55 ± 0.12	64.17 ± 0.27	64.96 ± 0.69
Hom./het.	NSD	**99.47 ± 0.53**	**99.69 ± 0.31**	73.60 ± 0.22	75.89 ± 0.49	33.65 ± 0.37	35.00 ± 0.21	72.30 ± 0.31	72.30 ± 0.10
	CSNN	**99.21 ± 0.79**	**99.40 ± 0.60**	76.64 ± 0.28	77.92 ± 0.07	35.15 ± 0.07	37.22 ± 0.13	74.11 ± 0.75	74.30 ± 0.80
Het./hom.	HAN	82.91 ± 0.61	85.00 ± 0.19	79.19 ± 0.82	82.22 ± 0.50	46.24 ± 0.06	45.02 ± 0.06	77.60 ± 0.43	77.50 ± 0.29
	PSHGCN	94.88 ± 0.10	95.00 ± 0.42	70.61 ± 0.13	72.32 ± 0.74	60.18 ± 0.24	58.33 ± 0.38	74.17 ± 0.34	74.96 ± 0.37
Het./het.	H2G-former	84.65 ± 0.71	85.00 ± 0.64	77.64 ± 0.83	81.25 ± 0.65	58.27 ± 0.21	58.33 ± 0.64	79.29 ± 0.61	79.32 ± 0.18
	Hetero^{2}Net	89.58 ± 0.16	90.00 ± 0.62	74.66 ± 0.69	78.57 ± 0.43	61.53 ± 0.21	61.36 ± 0.49	63.67 ± 0.66	65.45 ± 0.38
	H2SGNN	94.88 ± 0.09	95.00 ± 0.35	70.61 ± 0.37	72.32 ± 0.14	62.18 ± 0.44	61.31 ± 0.45	64.17 ± 0.06	64.96 ± 0.44
	HetSheaf	**100.00 ± 0.00**	**100.00 ± 0.00**	80.56 ± 0.00	82.14 ± 0.00	62.35 ± 0.00	63.33 ± 0.00	79.46 ± 0.00	79.46 ± 0.00
Ours	**H2S-NN**	**99.65 ± 0.35**	**99.72 ± 0.28**	**84.44 ± 0.21**	**86.23 ± 0.15**	**64.52 ± 0.56**	**65.10 ± 0.37**	**81.22 ± 0.06**	**81.52 ± 0.20**

Table 3: Performance on link prediction benchmarks measured by AUPR and AUROC. Best values are bold.

		LastFM		PubMed		Amazon		YouTube	
Group	**Model**	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC
Hom./hom.	GCN	97.06 ± 0.48	96.96 ± 0.41	96.05 ± 0.52	92.93 ± 0.36	92.40 ± 0.61	89.66 ± 0.73	89.30 ± 0.39	83.26 ± 0.52
	GAT	63.06 ± 0.27	51.14 ± 0.19	96.07 ± 0.43	93.27 ± 0.68	88.62 ± 0.34	86.10 ± 0.45	66.76 ± 0.31	58.40 ± 0.22
Hom./het.	NSD	97.16 ± 0.71	96.58 ± 0.65	96.41 ± 0.58	93.65 ± 0.74	92.63 ± 0.49	90.04 ± 0.38	89.32 ± 0.43	83.36 ± 0.57
	CSNN	97.99 ± 0.42	97.75 ± 0.39	96.14 ± 0.30	92.87 ± 0.29	92.10 ± 0.53	89.62 ± 0.41	88.64 ± 0.34	82.97 ± 0.48
Het./hom.	HAN	88.26 ± 0.37	83.32 ± 0.46	95.78 ± 0.49	93.62 ± 0.62	95.24 ± 0.56	93.64 ± 0.60	92.46 ± 0.51	90.10 ± 0.42
	PSHGCN	88.02 ± 0.44	80.55 ± 0.50	97.58 ± 0.35	93.27 ± 0.68	94.28 ± 0.46	92.34 ± 0.42	93.15 ± 0.63	90.66 ± 0.58
Het./het.	H2G-former	97.89 ± 0.51	97.93 ± 0.45	95.03 ± 0.77	93.15 ± 0.36	93.37 ± 0.40	91.42 ± 0.59	91.68 ± 0.65	88.33 ± 0.38
	Hetero^{2}Net	97.58 ± 0.29	97.45 ± 0.61	94.66 ± 0.53	92.57 ± 0.47	94.71 ± 0.54	92.34 ± 0.42	90.67 ± 0.70	86.45 ± 0.41
	H2SGNN	97.88 ± 0.36	97.22 ± 0.68	95.22 ± 0.42	92.32 ± 0.55	93.18 ± 0.61	92.31 ± 0.50	90.17 ± 0.46	88.96 ± 0.64
	LatGRL	96.88 ± 0.62	96.01 ± 0.71	96.61 ± 0.48	92.32 ± 0.52	94.41 ± 0.43	92.03 ± 0.57	89.98 ± 0.35	88.10 ± 0.49
	HetSheaf	98.51 ± 0.00	98.00 ± 0.00	96.36 ± 0.00	93.00 ± 0.00	93.76 ± 0.00	91.00 ± 0.00	91.27 ± 0.00	88.00 ± 0.00
Ours	**H2S-NN**	**98.91 ± 0.37**	**98.72 ± 0.35**	**97.63 ± 0.41**	**96.48 ± 0.54**	**98.53 ± 0.56**	**98.33 ± 0.44**	**93.95 ± 0.47**	**91.81 ± 0.33**

[8] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[9] Matthias Fey, Jinu Sunil, Akihiro Nitta, Rishi Puri, Manan Shah, Blaž Stojanovič, Ramona Bendias, Alexandria Barghi, Vid Kocijan, Zecheng Zhang, Xinwei He, Jan E. Lenssen, and Jure Leskovec. 2025. PyG 2.0: Scalable Learning on Real World Graphs. In *Temporal Graph Learning Workshop at KDD*.

[10] Stefano Fiorini, Hakan Aktas, Iulia Duta, Stefano Coniglio, Pietro Morerio, Alessio Del Bue, and Pietro Liò. 2025. Sheaves Reloaded: A Directional Awakening. *arXiv preprint arXiv:2506.02842* (2025).

[11] Karish Grover, Haiyang Yu, Qi Zhu, Han Xie, Vassilis Ioannidis, Christos Faloutsos, et al. 2025. Spectro-riemannian graph neural networks. In *International Conference on Learning Representations*, Vol. 2025. 57833–57859.

[12] Jiayan Guo, Lun Du, Wendong Bi, Qiang Fu, Xiaojun Ma, Xu Chen, Shi Han, Dongmei Zhang, and Yan Zhang. 2023. Homophily-oriented heterogeneous graph rewiring. In *Proceedings of the ACM web conference 2023*. 511–522.
[13] Jakob Hansen and Thomas Gebhart. 2020. Sheaf neural networks. *arXiv preprint arXiv:2012.06333* (2020).
[14] Mingguo He, Zhewei Wei, Shikun Feng, Zhengjie Huang, Weibin Li, Yu Sun, and Dianhai Yu. 2024. Spectral heterogeneous graph convolutions via positive noncommutative polynomials. In *Proceedings of the ACM Web Conference 2024*. 685–696.
[15] Mingguo He, Zhewei Wei, and Ji-Rong Wen. 2022. Convolutional Neural Networks on Graphs with Chebyshev Approximation, Revisited. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[16] Ferran Hernandez Caralt, Guillermo Bernárdez Gil, Iulia Duta, Pietro Liò, and Eduard Alarcón Cot. 2024. Joint Diffusion Processes as an Inductive Bias in Sheaf Neural Networks. In *Proceedings of the Geometry-grounded Representation Learning and Generative Modeling Workshop (GRaM) (Proceedings of Machine Learning Research, Vol. 251)*. PMLR, 249–263.
[17] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of the web conference 2020*. 2704–2710.
[18] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.
[19] Jintang Li, Zheng Wei, Yuchang Zhu, Ruofan Wu, Huizhe Zhang, Liang Chen, and Zibin Zheng. 2025. Heterophily-Aware Representation Learning on Heterogeneous Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 9 (2025), 7852–7866. doi:10.1109/TPAMI.2025.3573615
[20] Junhong Lin, Xiaojie Guo, Shuaicheng Zhang, Yada Zhu, and Julian Shun. 2025. When Heterophily Meets Heterogeneity: Challenges and a New Large-Scale Graph Benchmark. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 5607–5618.
[21] Xiaozhong Liu, Yingying Yu, Chun Guo, and Yizhou Sun. 2014. Meta-path-based ranking with pseudo relevance feedback on heterogeneous graph for citation recommendation. In *Proceedings of the 23rd acm international conference on conference on information and knowledge management*. 121–130.
[22] Kangkang Lu, Yanhua Yu, Ruopei Guo, Nan Cheng, Zhiyong Huang, Yunshan Ma, Meiyu Liang, Yuling Wang, Xiting Qin, Yimeng Ren, et al. 2026. Addressing graph heterogeneity and heterophily from a spectral perspective. *Neurocomputing* (2026), 133500.
[23] Qingsong Lv, Ming Ding, Qiang Liu, Yuxiang Chen, Wenzheng Feng, Siming He, Chang Zhou, Jianguo Jiang, Yuxiao Dong, and Jie Tang. 2021. Are we really making much progress? Revisiting, benchmarking, and refining heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. arXiv:2112.14936
[24] Qiheng Mao, Zemin Liu, Chenghao Liu, and Jianling Sun. 2023. Hinormer: Representation learning on heterogeneous information networks with graph transformer. In *Proceedings of the ACM web conference 2023*. 599–610.
[25] Alexander Mathiasen, Frederik Hvilshøj, Jakob Rødsgaard Jørgensen, Anshul Nasery, and Davide Mottin. 2020. What if neural networks had svds? *Advances in Neural Information Processing Systems* 33 (2020), 18411–18420.
[26] Christopher Morris, Nils M Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663* (2020).
[27] Emanuele Mule, Stefano Fiorini, Antonio Purificato, Federico Siciliano, Stefano Coniglio, and Fabrizio Silvestri. 2025. Directional Sheaf Hypergraph Networks: Unifying Learning on Directed and Undirected Hypergraphs. arXiv:2510.04727 [cs.LG] https://arxiv.org/abs/2510.04727
[28] Antonio Purificato, Giulia Cassarà, Federico Siciliano, Pietro Liò, and Fabrizio Silvestri. 2025. Sheaf4rec: Sheaf neural networks for graph-based recommender systems. *ACM Transactions on Recommender Systems* 4, 2 (2025), 1–26.
[29] André Ribeiro, Ana Luiza Tenorio, Juan Belieni, Amauri H Souza, and Diego Mesquita. 2026. Cooperative Sheaf Neural Networks. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=AHpexliCTM
[30] Zhixiang Shen and Zhao Kang. 2025. When heterophily meets heterogeneous graphs: Latent graphs guided unsupervised representation learning. *IEEE Transactions on Neural Networks and Learning Systems* (2025).
[31] Deepti Singh and Ankita Verma. 2025. An overview of heterogeneous social network analysis. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 15, 2 (2025), e70028.
[32] Yizhou Sun and Jiawei Han. 2013. Mining heterogeneous information networks: a structural analysis approach. *ACM SIGKDD explorations newsletter* 14, 2 (2013), 20–28.
[33] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S Yu, and Tianyi Wu. 2011. Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment* 4, 11 (2011), 992–1003.
[34] Zhen Tao, Ziyue Qiao, Chaoqi Chen, Zhengyi Yang, Lun Du, and Qingqiang Sun. 2025. On the Cross-type Homophily of Heterogeneous Graphs: Understanding and Unleashing. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 2842–2852.
[35] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
[36] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.
[37] Haosen Wang, Chenglong Shi, Can Xu, Surong Yan, Rong Xie, and Pan Tang. 2025. Enhancing Homophily in Heterogeneous Graph Contrastive Learning via Connection Strength and Multi-view Self-Expression. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Padua, Italy) *(SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 709–718. doi:10.1145/3726302.3729968
[38] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *The world wide web conference*. 2022–2032.
[39] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *The world wide web conference*. 2022–2032.
[40] Xiaocheng Yang, Mingyu Yan, Shirui Pan, Xiaochun Ye, and Dongrui Fan. 2023. Simple and efficient heterogeneous graph neural network. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 10816–10824.
[41] Xiaocheng Yang, Mingyu Yan, Shirui Pan, Xiaochun Ye, and Dongrui Fan. 2023. Simple and Efficient Heterogeneous Graph Neural Network. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
[42] Juwei Yue, Haikuo Li, Jiawei Sheng, Yihan Guo, Xinghua Zhang, Chuan Zhou, Tingwen Liu, and Li Guo. 2025. Graph wave networks. In *Proceedings of the ACM on Web Conference 2025*. 1365–1379.
[43] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 793–803.
[44] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An end-to-end deep learning architecture for graph classification. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.
[45] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.

A Data Preparation

A.1 Downstream Readouts

After sheaf-Hodge diffusion, the model has integrated information across both nodes and typed relations while retaining representations at the node and edge levels. Different prediction tasks, however, require this information to be summarized differently: node classification is centered on an individual target node, whereas link and graph prediction also depend on relational or graph-level structure. We therefore keep the sheaf-Hodge backbone shared across tasks and use a lightweight task-specific readout to convert the relevant final representations into logits.

Specifically, the backbone produces final node states $x^\star$, final edge states $s^\star$, and the initial typed node states $h^{(0)}$, which the corresponding decoder uses according to the downstream task.

Node classification. For node classification, each incident edge state is first transported into the stalk of its incident node before aggregation. Let $V_{\text{target}} \subseteq V$ denote the target-node set. For each node v, define

$$\widetilde{s}_v^\star = \begin{cases} \dfrac{1}{d_v}\left(\displaystyle\sum_{\mathrm{src}(e)=v} F_{\mathrm{src},e}^{\mathsf{T}} s_e^\star + \sum_{\mathrm{tgt}(e)=v} F_{\mathrm{tgt},e}^{\mathsf{T}} s_e^\star\right), & d_v > 0, \\ 0, & d_v = 0. \end{cases} \tag{29}$$

Here $d_v := \deg_{\mathrm{ext}}(v)$ counts incidences from both observed and closure edges. The transpose restriction maps transport each edge state from its edge stalk into the corresponding node stalk before averaging. Both endpoint contributions use a positive sign because

$\widetilde{s_v^\star}$ is an incident-edge context summary rather than the orientation-sensitive coboundary adjoint $\delta_0^\top s^\star$. The node logits are then

$$\ell_v = \mathrm{MLP}_{\mathrm{node}}\left(x_v^\star \,\|\, h_v^{(0)} \,\|\, \widetilde{s_v^\star}\right), \qquad v \in V_{\mathrm{target}}. \tag{30}$$

Link prediction. For link prediction, we reuse the transported context $\widetilde{s_v^\star}$ from Equation (29) and concatenate it with the final node state:

$$z_v = \mathrm{GELU}\big(W_{\mathrm{link}}[x_v^\star \,\|\, \widetilde{s_v^\star}] + b_{\mathrm{link}}\big) \in \mathbb{R}^d.$$

This readout makes the link-prediction loss depend on the edge cochain while combining only vectors expressed in the same endpoint node stalk. Given a candidate pair (u, v), the dot-product decoder returns

$$\ell_{uv}^{\mathrm{dot}} = z_u^\top z_v. \tag{31}$$

The relation-specific bilinear branch, configured as `distmult`, uses one full matrix $W_r \in \mathbb{R}^{d\times d}$ for each scored relation type r:

$$\ell_{uvr}^{\mathrm{bilinear}} = z_u^\top W_r z_v. \tag{32}$$

The concatenation branch instead applies a small MLP,

$$\ell_{uv}^{\mathrm{concat}} = \mathrm{MLP}_{\mathrm{link}}(z_u \,\|\, z_v). \tag{33}$$

All three branches return raw logits and are trained with binary cross-entropy with logits. The decoder family is fixed by the dataset configuration unless it is varied in the dedicated decoder-family ablation.

Graph classification. For graph classification, SheafCovPool converts the node and edge cochains of a graph g into a summary vector γ_g. The attention-weighted means and covariances of the Gram-matrix features make this vector invariant to node and edge permutations and to independent orthogonal changes of stalk basis. The graph classifier is then a final MLP:

$$\ell_g = \mathrm{MLP}_{\mathrm{graph}}(\gamma_g). \tag{34}$$

The logits ℓ_g are trained with cross-entropy for single-label graph classification, or binary cross-entropy with logits for multilabel graph classification.

B Hyperparameter Tuning

The registered primary tuning protocol uses W&B Bayesian search with Hyperband early termination and the task-specific validation monitor. Node and link prediction use one complete-graph forward/backward update per epoch; query minibatching is not part of this initial protocol. Graph classification uses the implementation's fixed batch size of 32 independent graphs while retaining each graph as a complete complex. Optimizer family, scheduler, gradient clipping, data splits, and AMPSS Stage-I support are held fixed. Early-stopping patience is fixed within each sweep and is supplied by the experiment configuration or command. Table 6 lists only the active primary search dimensions.

Encoder dropout is omitted for the all-featureless identity-embedding fallback, and decoder dropout is omitted for dot-product and DistMult link decoders. Optional pooling-family, decoder-family, and AMPSS operator-ramp sweeps are treated as ablations rather than being mixed into the primary search. Changing the stalk dimension does not silently rewrite the restriction rank or the edge, face, and classifier hidden widths stored in the base configuration.

C Benchmark Datasets

We evaluated the models on benchmark datasets covering node classification, link prediction, and graph classification. The node- and link-prediction datasets follow the Heterogeneous Graph Benchmark (HGB) [23]. For graph classification, we used datasets from TUDataset [26]; node and edge labels are treated as node and relation types to instantiate heterogeneous graph inputs [5].

C.1 Node Classification

DBLP. DBLP is a bibliographic network containing four types of entities: authors, papers, venues, and terms. These entities are connected through six relation types, including authorship, publication-venue, paper-term, and author-venue relations. The prediction target is the author node type, where each author is assigned to one of four research areas: database systems, data mining, artificial intelligence, or information retrieval.

IMDB. IMDB is a heterogeneous movie network composed of movies, directors, actors, and keywords. Its six relation types represent relations such as movie-director, movie-actor, and movie-keyword associations. The task is multilabel movie classification, in which a movie may belong to one or more of five genres: action, comedy, drama, romance, and thriller.

ACM. ACM is an academic citation network with four node types: papers, authors, subjects, and terms. Its relations include paper citations, authorship, paper-subject associations, and paper-term associations. The objective is to classify papers into three research categories: database systems, wireless communication, and data mining.

C.2 Link Prediction

LastFM. LastFM is a music-oriented heterogeneous network containing users, artists, and tags. The graph includes user-artist, user-user, and artist-tag relations. The link-prediction task determines whether an interaction exists between a user and an artist.

PubMed. PubMed is a biomedical knowledge graph containing genes, diseases, chemicals, and species. Ten relation types describe biomedical associations, including gene-disease, chemical-chemical, and species-disease relations. Evaluation focuses on predicting links between disease nodes.

YouTube. YouTube is a social interaction graph containing 2,000 user nodes connected by five distinct types of user interactions. The prediction task is to infer missing user-user links.

Amazon. Amazon is a product interaction graph with one node type and two relation types. The task is to predict missing product-product links.

C.3 Graph Classification

MUTAG. MUTAG contains 188 molecular graphs representing aromatic and heteroaromatic nitro compounds. Nodes correspond to seven atom types, while edges encode four bond types. Each graph is classified according to whether the corresponding compound exhibits a mutagenic effect.

Table 6: Registered primary hyperparameter search space. Conditional entries are included only when the named module consumes them.

Type	Parameter	Set of parameters
H2SNN	Stalk dimension	node/link: {16, 32}; graph: {16, 32, 64}
	Hodge-diffusion steps (K)	{2, 3, 4, 5, 6}
	Node-to-edge diffusion step size	log-uniform on [0.005, 0.3]
	Edge-to-face diffusion step size	log-uniform on [0.005, 0.3]
	Layer dropout	{0.0, 0.1, 0.2, 0.3}
	Input dropout	{0.0, 0.1, 0.2, 0.3}
	Classifier/head dropout	{0.0, 0.1, 0.2, 0.3} when consumed by the head
	Learning rate	$\{10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}, 5\times10^{-3}, 10^{-2}, 5\times10^{-2}\}$
	Weight decay	$\{0, 10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}, 5\times10^{-3}\}$
SheafCovPool	Raw Gram dimension	fixed at 3×3 (three semantic views)
	Post-Gram feature dimension F	{4, 8, 16}

Table 7: Node-classification dataset statistics and semantic meta-path schemas. The #N and #E columns indicate nodes and edges.

Dataset	#N	#N types	#E	#R types	#Classes	Target	Meta-path schemas
DBLP	26,128	4	239,566	6	4	author	APA, APCPA, APTPA
IMDB	21,420	4	86,642	6	5	movie	MAM, MDM
ACM	10,942	4	547,806	7	3	paper	PAP, PSP

Table 8: Heterogeneous link-prediction dataset statistics. The #N and #E columns indicate nodes and edges.

Dataset	#N	#N types	#E	#R types	Target
LastFM	20,612	3	141,521	3	user-artist
PubMed	63,109	4	244,986	10	disease-disease
YouTube	2,000	1	1,177,130	5	user-user
Amazon	10,099	1	148,659	2	product-product

Table 9: Graph-classification dataset statistics. The Graphs column indicates the number of graphs, and #E indicates edges.

Dataset	#Graphs	#N types	#E	#R types	Target
MUTAG	188	7	7,442	4	mutagenicity
PROTEINS	1,113	3	162,088	1	enzyme/non-enzyme
ENZYMES	600	3	19,580	1	enzyme class
NCI1	4,110	37	49,205	1	anti-cancer activity

PROTEINS. PROTEINS consists of 1,113 protein graphs. Nodes represent secondary-structure elements belonging to three types, and edges connect elements that are adjacent either in the amino-acid sequence or in three-dimensional space. The task is binary classification of proteins into enzyme and non-enzyme classes.

ENZYMES. ENZYMES contains 600 graph representations of protein tertiary structures collected from the BRENDA enzyme database. Its nodes describe three types of secondary-structure elements. Each graph is assigned to one of the six top-level Enzyme Commission classes.

NCI1. NCI1 comprises 4,110 chemical compounds evaluated for their ability to inhibit the growth of human tumor cell lines. Molecular nodes represent 37 atom types. The graph-classification objective is to determine whether each compound exhibits anti-cancer activity.

D Additional H2S-NN Experiments: Node Clustering

Although the main experiments evaluate supervised graph-learning tasks, we also used clustering to examine whether the learned representations preserve class-relevant local structure. Following the evaluation procedure used for Heterogeneous Graph Attention Networks [39], we ran k-means on the node embeddings after supervised training. We used DBLP with $k = 4$ and ACM with $k = 3$, matching the number of classes in each dataset, and set the maximum number of iterations to 300.

We compared H2S-NN against four groups of baselines: (i) homogeneous homophilic models, including GAT [36]; (ii) homogeneous heterophilic models, including NSD [4] and CSNN [29]; (iii) heterogeneous homophilic models, including HAN [39] and PSHGCN [14]; and (iv) heterogeneous heterophilic models, including H2G-former [20], Hetero^{2}Net [19], and LatGRL [30].

We used normalized mutual information (NMI) and the adjusted Rand index (ARI) to evaluate clustering quality. We repeated the clustering step over five random seeds and reported the mean and standard deviation. As shown in Table 10, H2S-NN achieved the highest mean NMI and ARI on both DBLP and ACM. These results suggest that the embeddings learned by H2S-NN are more separable under this clustering diagnostic. The t-SNE visualizations in Figures 5 and 6 provide a qualitative view of these embeddings [35].

E Visualization of Node Embedding Separability During Training

We further inspected the training dynamics of H2S-NN on the controlled four-class `FakeHeteroDataset` from PyTorch Geometric [8, 9]. The model was trained with the default synthetic-data settings, and the visualization includes only labeled nodes. We



Figure 5: t-SNE visualization of learned ACM paper embeddings. Each point represents one paper, and color denotes its class.



Figure 6: t-SNE visualization of learned DBLP author embeddings. Each point represents one author, and color denotes its research area.

Table 10: Additional clustering-quality diagnostics on DBLP and ACM, reported as mean ± standard deviation over five runs. Best mean values are bold.

		DBLP		ACM	
Group	**Model**	NMI	ARI	NMI	ARI
Hom./hom.	GAT	71.50 ± 0.36	77.26 ± 0.48	57.22 ± 0.19	60.47 ± 0.31
Hom./het.	NSD	80.21 ± 0.62	84.04 ± 0.43	62.56 ± 0.37	65.02 ± 0.28
	CSNN	81.72 ± 0.39	84.25 ± 0.55	69.28 ± 0.45	71.10 ± 0.34
Het./hom.	HAN	79.12 ± 0.47	84.76 ± 0.58	60.63 ± 0.22	64.28 ± 0.41
	PSHGCN	80.38 ± 0.52	82.65 ± 0.36	71.89 ± 0.49	74.31 ± 0.57
Het./het.	Hetero^{2}Net	80.54 ± 0.33	84.98 ± 0.46	70.14 ± 0.40	73.42 ± 0.51
	LatGRL	81.36 ± 0.56	86.07 ± 0.38	72.52 ± 0.61	76.75 ± 0.44
Ours	**H2S-NN**	**82.41 ± 0.35**	**86.52 ± 0.42**	**75.78 ± 0.37**	**78.29 ± 0.31**

recorded the node embeddings every ten epochs. As shown in Figure 7, nodes from different classes are strongly interleaved at the beginning of training. As training proceeds, the projected embeddings of nodes in the same class form increasingly compact and separated regions. This observation suggests that training H2S-NN produces more class-separable node representations on this controlled dataset.

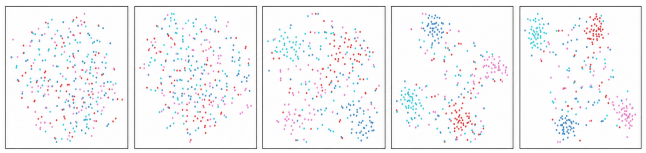


Figure 7: Qualitative visualization of node embedding dynamics during training on the four-class `FakeHeteroDataset`.

F Computational Efficiency

To characterize the computational cost, we analyze the H2S-NN model, the AMPSS component, and the SheafCovPool readout.

Let $N = |V|$ and $M = |E|$ denote the numbers of nodes and observed directed edges. Let N_{inst} be the number of retained concrete path instances. Define the number of boundary incidences by

$$I_{\text{bnd}} = \sum_{f \in \mathcal{I}_{\text{ret}}} (\ell_f + 1).$$

Each instance contributes one closure 1-cell in addition to its path edges. Let $\widetilde{M} = M + N_{\text{inst}}$ denote the size of the extended edge domain. We use d for the stalk dimension and K for the number of diffusion steps.

AMPSS.. Stage I runs once before training. Let C_Σ be the number of viable schemas, $S_{\max}$ the selection limit, N_p the number of probe examples, B the feature width per schema, C_y the number of classes, and R_{sel} the number of probe iterations. Greedy selection evaluates $O(C_\Sigma S_{\max})$ subsets whose feature width grows to $S_{\max}B$, giving

$$O(R_{\text{sel}} N_p B C_y C_\Sigma S_{\max}^2)$$

probe time. Label propagation and schema/path enumeration additionally cost $O(K_{\text{LP}}(M+N)C_y + Q_{\text{type}} + Q_{\text{samp}})$, where K_{LP} is the number of label-propagation iterations and the Q terms count the visited typed and concrete path prefixes. Sparse coverage and schema features require

$$O(C_\Sigma L_{\max} + N_p S_{\max} B + N_{\text{cov}} + I_{\text{bnd}})$$

storage, where $L_{\max}$ is the maximum schema length and N_{cov} is the number of stored coverage entries. Stage II adds $O(N_{\text{inst}}(h_f + d_r)h_\beta)$ instance-weighting time, where h_f is the ordered-path latent width and h_β is the weighting-network width. The positive weights do not prune operator tensors.

H2S-NN.. To construct the scaled-orthogonal restriction maps, we use the Fast Householder Product parameterization [25]. Node-to-edge maps are learned only for the M observed edges, whereas closure and edge-to-face maps are obtained by path transport. Their dense $d \times d$ matrix operations require $O((M + I_{\text{bnd}})d^3)$ construction time. The resulting maps are cached across diffusion steps, requiring $O((\widetilde{M} + I_{\text{bnd}})d^2)$ space. Thus, map construction is cubic in the stalk dimension, while memory grows quadratically. Each matrix-free diffusion step applies the node-mediated and face-induced components of the edge operator in

$$O\Big((\widetilde{M} + I_{\text{bnd}})d^2 + (N + \widetilde{M})d\Big)$$

time. Hence, when the MLP widths and d_r scale with d, the backbone costs

$$O\Big(N d_{\text{in}} d + (M + I_{\text{bnd}})d^3 + K\big[(\widetilde{M} + I_{\text{bnd}})d^2 + (N + \widetilde{M})d\big]\Big)$$

time and

$$O\Big((\widetilde{M} + I_{\text{bnd}})d^2 + (N + \widetilde{M} + I_{\text{bnd}})d\Big)$$

inference space. For fixed d and K, both costs are linear in the graph size and retained boundary incidences; no dense Laplacian is formed.

SheafCovPool. With three node/edge views, projected dimension d_p, pooling width h_p, N_{graph} graphs, and C_{cls} classes, the readout costs

$$O\Big(\widetilde{M}d^2 + (N + \widetilde{M})(d + h_p d_p + d_p^2) + N_{\text{graph}}(h_p d_p^2 + h_p C_{\text{cls}})\Big)$$

time and

$$O\Big((N + \widetilde{M})(d + d_p^2) + N_{\text{graph}} d_p^2\Big)$$

space. The $\widetilde{M}d^2$ term transports extended-edge states into their endpoint stalks. With fixed dimensions, SheafCovPool is linear in the extended graph size and constructs neither a dense node-to-node covariance matrix nor a coarsened graph.

Table 11 reports the measured runtime, parameter count, and memory usage on the ACM node-classification task.

Table 11: Efficiency comparison on ACM. Best values are bold. HetSheaf's code is unavailable, so we use the results reported for the HetSheaf ensemble model in the original paper.

Model	Runtime (s)	Parameters	Memory (MB)
HAN	28.9	2.2M	1,780
HGT	1,025.8	7.2M	6,120
NSD	47.6	**155K**	**980**
HetSheaf	55.7	174K	-
H2S-NN	72.1	517K	1120

Table 11 shows that H2S-NN requires more parameters, more memory, and has a higher runtime than other models including HetSheaf, the closest heterogeneous sheaf baseline. This is because H2S-NN retains path-supported $2D$-cells and performs sheaf-Hodge diffusion on both node and edge cochains, which increases the computational cost. However, H2S-NN achieves better performance than all baselines, as shown in Table 1, Table 3, and Table 2. This suggests that the additional computational cost of H2S-NN is justified by its superior performance.